RAMAIAH UNIVERSITY OF APPLIED SCIENCES

SCHOOL OF SOCIAL SCIENCES

Department of Data Sciences And Analytics

Course Name: DBMS

Course Code: DSC202A

Name: ABHIJITH S

Semester/Batch: 3rd sem / 2025

Registration No: 25SSDS415002

Year: 2026

CINEMA ANALYTICS

Director Filmography & Box Office Tracker

Objective

To create a simple cinema management application that stores and manages details about directors, movies and box-office collections. The application allows users to add, view, edit and delete records. It also brings related information together using SQL JOIN operations.

This project is a small system for storing cinema information. It keeps details about directors, their movies and the box-office collections of those movies. The information is stored in a database, and a simple window is provided so users can work with the records without writing SQL commands.

Tools Used

1. Python: Used to write the main program and control how the application works.
2. SQLite: Used to create the database and store records in related tables.
3. Tkinter: Used to create the application window, tabs, forms, buttons and records table.
4. sqlite3: A Python module that connects the program to the SQLite database and runs SQL commands.

Python and SQLite

SQL is used to create tables, store data, search for records, update records and connect related tables. However, SQL alone is mainly used to work with the database. It does not provide the complete desktop application interface used in this project.

Python was used to control the application and create the user interface with Tkinter. SQLite was used to store the data. The Python sqlite3 module connects Python to SQLite and sends SQL commands to it. This allows users to work through a simple graphical window while the database operations are still performed using SQL.

So, the project was not done without SQL. SQL is used inside the Python program for database operations, while Python

manages the interface and the flow of the application. Using only SQL would not provide the same ready-to-use graphical interface.

Main Features

- Stores director names and nationalities.
- Stores movie titles, release years and director details.
- Stores box-office revenue and territory details.
- Allows users to add, view, edit and delete records.
- Uses primary keys and foreign keys to connect the tables.
- Uses JOIN operations to show director, movie and box-office details together.
- Checks user input and displays messages for missing or invalid entries.

Python, SQL and Tkinter

Python is the main language used to write this application. Some ready-made Python modules are imported so that the program can perform different tasks. Importing means making these modules available for use in the program.

The sqlite3 module is imported to connect Python to the SQLite database. It allows the Python program to send SQL commands to the database. For example, SQL commands are used to insert new records, display records, update details and delete records. The database stores the information, while SQL is used to work with that information.

The tkinter module is imported to create the graphical user interface (GUI). A GUI is the window that users see and interact with. Tkinter is used to make the tabs, text boxes, buttons, drop-down lists and table shown in this project. Users can enter or select information using these items instead of typing database commands.

In simple terms, Python controls the application, Tkinter creates the window and its controls, and sqlite3 connects Python to SQLite. SQL commands are sent through that connection to store, find, change and remove database records.

Working

1. The program connects to the cinema.db database using sqlite3.
2. It creates the Directors, Movies and Box Office tables if they do not already exist.
3. Sample records are added when the database is empty.
4. Tkinter displays the application window with separate tabs for different tasks.
5. When a user enters information and clicks a button, Python reads the input and sends the required SQL command to SQLite.
6. The database saves or returns the information, and the application refreshes the records shown on the screen.

Code

```
# ============================================================
# CINEMA ANALYTICS
# Director Filmography & Box Office Tracker
# Complete single-cell version for Jupyter Notebook
# ============================================================

# IMPORTANT:
# Run this entire code in ONE Jupyter cell.
# Do not run the old GUI cells separately. This code creates
# one Tkinter window and starts its event loop only once.

import sqlite3
import tkinter as tk
from tkinter import ttk, messagebox

# -------------------- DATABASE CONNECTION --------------------

# Create or connect to cinema.db in the Jupyter working folder.
database = sqlite3.connect("cinema.db")

# Enable foreign-key rules, including ON DELETE CASCADE.
database.execute("PRAGMA foreign_keys = ON")
cursor = database.cursor()

# -------------------- CREATE DATABASE TABLES --------------------

cursor.execute("""
CREATE TABLE IF NOT EXISTS directors (
    director_id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    nationality TEXT
)
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS movies (
```

```
    movie_id INTEGER PRIMARY KEY AUTOINCREMENT,
    director_id INTEGER,
    title TEXT NOT NULL,
    release_year INTEGER,
    FOREIGN KEY (director_id)
        REFERENCES directors(director_id)
        ON DELETE CASCADE
)
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS box_office (
    record_id INTEGER PRIMARY KEY AUTOINCREMENT,
    movie_id INTEGER,
    revenue_millions REAL,
    territory TEXT,
    FOREIGN KEY (movie_id)
        REFERENCES movies(movie_id)
        ON DELETE CASCADE
)
""")

# -------------------- INSERT INITIAL SAMPLE DATA --------------------

# Sample records are inserted only when the database has no directors.
cursor.execute("SELECT COUNT(*) FROM directors")

if cursor.fetchone()[0] == 0:
    directors_data = [
        ("Christopher Nolan", "British-American"),
        ("S. S. Rajamouli", "Indian"),
        ("Steven Spielberg", "American"),
        ("James Cameron", "Canadian")
    ]

    cursor.executemany(
        "INSERT INTO directors (name, nationality) VALUES (?, ?)",
        directors_data
    )

    movies_data = [
        (1, "Inception", 2010),
        (1, "Interstellar", 2014),
        (2, "RRR", 2022),
        (2, "Baahubali", 2015),
        (3, "Jurassic Park", 1993),
        (4, "Avatar", 2009)
    ]

    cursor.executemany(
        "INSERT INTO movies (director_id, title, release_year) VALUES (?, ?, ?)",
        movies_data
    )

    box_office_data = [
        (1, 839.0, "Worldwide"),
        (2, 731.0, "Worldwide"),
        (3, 160.0, "Worldwide"),
        (4, 650.0, "Worldwide"),
        (5, 1100.0, "Worldwide"),
```

```
        (6, 2923.0, "Worldwide")
    ]

    cursor.executemany(
        """INSERT INTO box_office (movie_id, revenue_millions, territory)
           VALUES (?, ?, ?)""",
        box_office_data
    )

database.commit()


# -------------------- DATABASE OPERATIONS --------------------
# These functions are used by the GUI to insert, retrieve,
# update and delete records in the SQLite database.

def add_director(name, nationality):
    cursor.execute(
        "INSERT INTO directors (name, nationality) VALUES (?, ?)",
        (name, nationality)
    )
    database.commit()



def add_movie(director_id, title, release_year):
    cursor.execute(
        """INSERT INTO movies (director_id, title, release_year)
           VALUES (?, ?, ?)""",
        (director_id, title, release_year)
    )
    database.commit()



def add_box_office(movie_id, revenue, territory):
    cursor.execute(
        """INSERT INTO box_office (movie_id, revenue_millions, territory)
           VALUES (?, ?, ?)""",
        (movie_id, revenue, territory)
    )
    database.commit()



def get_director_options():
    cursor.execute(
        "SELECT director_id, name FROM directors ORDER BY name"
    )
    return [f"{row[0]} - {row[1]}" for row in cursor.fetchall()]



def get_movie_options():
    cursor.execute(
        "SELECT movie_id, title FROM movies ORDER BY title"
    )
    return [f"{row[0]} - {row[1]}" for row in cursor.fetchall()]



def get_filmography():
    # JOIN combines directors, movies and box-office records.
```

```
    # movie_id is included internally to identify each movie row.
    cursor.execute("""
        SELECT
            movies.movie_id,
            directors.name,
            movies.title,
            movies.release_year,
            box_office.revenue_millions,
            box_office.territory
        FROM directors
        INNER JOIN movies
            ON directors.director_id = movies.director_id
        LEFT JOIN box_office
            ON movies.movie_id = box_office.movie_id
        ORDER BY directors.name, movies.release_year
    """)
    return cursor.fetchall()



def update_director(director_id, name, nationality):
    cursor.execute(
        """UPDATE directors
           SET name = ?, nationality = ?
           WHERE director_id = ?""",
        (name, nationality, director_id)
    )
    if cursor.rowcount == 0:
        raise ValueError("Director ID was not found.")
    database.commit()



def update_movie(movie_id, director_id, title, release_year):
    cursor.execute(
        """UPDATE movies
           SET director_id = ?, title = ?, release_year = ?
           WHERE movie_id = ?""",
        (director_id, title, release_year, movie_id)
    )
    if cursor.rowcount == 0:
        raise ValueError("Movie ID was not found.")
    database.commit()



def update_box_office(record_id, movie_id, revenue, territory):
    cursor.execute(
        """UPDATE box_office
           SET movie_id = ?, revenue_millions = ?, territory = ?
           WHERE record_id = ?""",
        (movie_id, revenue, territory, record_id)
    )
    if cursor.rowcount == 0:
        raise ValueError("Box-office record ID was not found.")
    database.commit()



def delete_movie(movie_id):
    cursor.execute(
        "DELETE FROM movies WHERE movie_id = ?",
        (movie_id,)
    )
```

```
    if cursor.rowcount == 0:
        raise ValueError("Movie record was not found.")
    database.commit()



def extract_id(value):
    return int(value.split(" - ", 1)[0])



# -------------------- GRAPHICAL USER INTERFACE --------------------
# The complete GUI is created once below. Do not create another
# Tk() window in a separate cell while this application is running.



# ============================================================
# CINEMA ANALYTICS - FINAL GUI
# Add, View, Edit and Delete Directors, Movies and Box Office
# ============================================================

import tkinter as tk
from tkinter import ttk, messagebox
import sqlite3


# Reuse the existing database connection
cursor.execute("PRAGMA foreign_keys = ON")


# -------------------- WINDOW SETUP --------------------


root = tk.Tk()
root.title("Cinema Analytics | Filmography & Box Office Tracker")
root.geometry("1100x700")
root.configure(bg="#F3F6FC")


NAVY = "#14213D"
BLUE = "#2563EB"
LIGHT = "#F3F6FC"
WHITE = "#FFFFFF"
TEXT = "#1E293B"
MUTED = "#64748B"


style = ttk.Style()
style.theme_use("clam")


style.configure(
    "TNotebook",
    background=LIGHT,
    borderwidth=0
)


style.configure(
    "TNotebook.Tab",
    padding=(22, 12),
    font=("Arial", 10, "bold")
)


style.configure(
```

```
    "Treeview",
    background=WHITE,
    foreground=TEXT,
    rowheight=30,
    fieldbackground=WHITE,
    font=("Arial", 10)
)

style.configure(
    "Treeview.Heading",
    background=NAVY,
    foreground=WHITE,
    font=("Arial", 10, "bold")
)

style.map(
    "Treeview",
    background=[("selected", "#DBEAFE")],
    foreground=[("selected", TEXT)]
)

# -------------------- HELPER FUNCTIONS --------------------

def make_label(parent, text):
    return tk.Label(
        parent,
        text=text,
        bg=WHITE,
        fg=TEXT,
        font=("Arial", 10, "bold")
    )

def make_entry(parent):
    return tk.Entry(
        parent,
        font=("Arial", 11),
        relief="solid",
        bd=1
    )

def make_button(parent, text, command, color=BLUE):
    return tk.Button(
        parent,
        text=text,
        command=command,
        bg=color,
        fg=WHITE,
        activebackground=NAVY,
        activeforeground=WHITE,
        font=("Arial", 10, "bold"),
        relief="flat",
        padx=15,
        pady=8,
        cursor="hand2"
    )

def get_director_options():
    cursor.execute(
        "SELECT director_id, name FROM directors ORDER BY name"
    )
    return [
```

```
        f"{row[0]} - {row[1]}"
        for row in cursor.fetchall()
    ]

def get_movie_options():
    cursor.execute(
        "SELECT movie_id, title FROM movies ORDER BY title"
    )
    return [
        f"{row[0]} - {row[1]}"
        for row in cursor.fetchall()
    ]

def extract_id(value):
    return int(value.split(" - ")[0])

# -------------------- HEADER --------------------

header = tk.Frame(root, bg=NAVY, height=105)
header.pack(fill="x")

tk.Label(
    header,
    text="CINEMA ANALYTICS",
    bg=NAVY,
    fg=WHITE,
    font=("Arial", 23, "bold")
).pack(anchor="w", padx=30, pady=(20, 2))

tk.Label(
    header,
    text="DIRECTOR FILMOGRAPHY & BOX OFFICE TRACKER",
    bg=NAVY,
    fg="#BFDBFE",
    font=("Arial", 10)
).pack(anchor="w", padx=32)

# -------------------- NOTEBOOK --------------------

notebook = ttk.Notebook(root)
notebook.pack(fill="both", expand=True, padx=20, pady=20)

director_tab = tk.Frame(notebook, bg=WHITE)
movie_tab = tk.Frame(notebook, bg=WHITE)
box_tab = tk.Frame(notebook, bg=WHITE)
records_tab = tk.Frame(notebook, bg=WHITE)

notebook.add(director_tab, text="DIRECTORS")
notebook.add(movie_tab, text="MOVIES")
notebook.add(box_tab, text="BOX OFFICE")
notebook.add(records_tab, text="DATABASE RECORDS")

# -------------------- DIRECTOR TAB --------------------

tk.Label(
    director_tab,
    text="ADD NEW DIRECTOR",
    bg=WHITE,
```

```
    fg=NAVY,
    font=("Arial", 17, "bold")
).pack(anchor="w", padx=35, pady=(35, 20))

director_form = tk.Frame(director_tab, bg=WHITE)
director_form.pack(anchor="w", padx=35)

make_label(director_form, "Director Name").grid(
    row=0, column=0, sticky="w", pady=8
)
director_name = make_entry(director_form)
director_name.grid(row=0, column=1, padx=20, pady=8)

make_label(director_form, "Nationality").grid(
    row=1, column=0, sticky="w", pady=8
)
director_nationality = make_entry(director_form)
director_nationality.grid(row=1, column=1, padx=20, pady=8)

def add_director_gui():
    name = director_name.get().strip()
    nationality = director_nationality.get().strip()

    if not name:
        messagebox.showwarning("Input Required", "Enter the director's name.")
        return

    try:
        add_director(name, nationality)
        messagebox.showinfo("Success", "Director added successfully.")

        director_name.delete(0, tk.END)
        director_nationality.delete(0, tk.END)

        refresh_dropdowns()
        refresh_table()

    except Exception as error:
        messagebox.showerror("Error", str(error))

make_button(
    director_tab,
    "ADD DIRECTOR",
    add_director_gui
).pack(anchor="w", padx=35, pady=25)

# -------------------- MOVIE TAB --------------------

tk.Label(
    movie_tab,
    text="ADD NEW MOVIE",
    bg=WHITE,
    fg=NAVY,
    font=("Arial", 17, "bold")
).pack(anchor="w", padx=35, pady=(30, 20))

movie_form = tk.Frame(movie_tab, bg=WHITE)
```

```
movie_form.pack(anchor="w", padx=35)


make_label(movie_form, "Movie Title").grid(
    row=0, column=0, sticky="w", pady=8
)
movie_title = make_entry(movie_form)
movie_title.grid(row=0, column=1, padx=20, pady=8)


make_label(movie_form, "Release Year").grid(
    row=1, column=0, sticky="w", pady=8
)
movie_year = make_entry(movie_form)
movie_year.grid(row=1, column=1, padx=20, pady=8)


make_label(movie_form, "Director").grid(
    row=2, column=0, sticky="w", pady=8
)
movie_director = ttk.Combobox(
    movie_form,
    width=27,
    state="readonly"
)
movie_director.grid(row=2, column=1, padx=20, pady=8)


def add_movie_gui():
    title = movie_title.get().strip()
    year = movie_year.get().strip()
    director = movie_director.get()


    if not title or not year or not director:
        messagebox.showwarning("Input Required", "Complete all movie fields.")
        return


    try:
        year = int(year)
        if year <= 0:
            raise ValueError
        director_id = extract_id(director)


        add_movie(director_id, title, year)


        messagebox.showinfo("Success", "Movie added successfully.")


        movie_title.delete(0, tk.END)
        movie_year.delete(0, tk.END)


        refresh_dropdowns()
        refresh_table()


    except ValueError:
        messagebox.showerror("Invalid Input", "Enter a valid release year.")
    except Exception as error:
        messagebox.showerror("Error", str(error))


make_button(
    movie_tab,
    "ADD MOVIE",
    add_movie_gui
```

```
).pack(anchor="w", padx=35, pady=25)

# -------------------- BOX OFFICE TAB --------------------

tk.Label(
    box_tab,
    text="ADD BOX OFFICE RECORD",
    bg=WHITE,
    fg=NAVY,
    font=("Arial", 17, "bold")
).pack(anchor="w", padx=35, pady=(30, 20))

box_form = tk.Frame(box_tab, bg=WHITE)
box_form.pack(anchor="w", padx=35)

make_label(box_form, "Movie").grid(
    row=0, column=0, sticky="w", pady=8
)
box_movie = ttk.Combobox(
    box_form,
    width=27,
    state="readonly"
)
box_movie.grid(row=0, column=1, padx=20, pady=8)

make_label(box_form, "Revenue (Millions)").grid(
    row=1, column=0, sticky="w", pady=8
)
box_revenue = make_entry(box_form)
box_revenue.grid(row=1, column=1, padx=20, pady=8)

make_label(box_form, "Territory").grid(
    row=2, column=0, sticky="w", pady=8
)
box_territory = make_entry(box_form)
box_territory.grid(row=2, column=1, padx=20, pady=8)

def add_box_gui():
    movie = box_movie.get()
    revenue = box_revenue.get().strip()
    territory = box_territory.get().strip()

    if not movie or not revenue or not territory:
        messagebox.showwarning("Input Required", "Complete all box office fields.")
        return

    try:
        movie_id = extract_id(movie)
        revenue = float(revenue)

        if revenue < 0:
            raise ValueError

        add_box_office(movie_id, revenue, territory)

        messagebox.showinfo("Success", "Box office record added successfully.")
```

```
        box_revenue.delete(0, tk.END)
        box_territory.delete(0, tk.END)

        refresh_table()

    except ValueError:
        messagebox.showerror(
            "Invalid Input",
            "Enter a valid non-negative revenue value."
        )
    except Exception as error:
        messagebox.showerror("Error", str(error))

make_button(
    box_tab,
    "ADD BOX OFFICE RECORD",
    add_box_gui
).pack(anchor="w", padx=35, pady=25)

# -------------------- DATABASE RECORDS TAB --------------------

tk.Label(
    records_tab,
    text="COMPLETE FILMOGRAPHY",
    bg=WHITE,
    fg=NAVY,
    font=("Arial", 17, "bold")
).pack(anchor="w", padx=20, pady=(20, 10))

table_frame = tk.Frame(records_tab, bg=WHITE)
table_frame.pack(fill="both", expand=True, padx=20, pady=10)

columns = (
    "Director",
    "Movie",
    "Year",
    "Revenue",
    "Territory"
)

tree = ttk.Treeview(
    table_frame,
    columns=columns,
    show="headings",
    height=12
)

for column in columns:
    tree.heading(column, text=column)
    tree.column(column, width=150, anchor="center")

tree.pack(side="left", fill="both", expand=True)

scrollbar = ttk.Scrollbar(
    table_frame,
    orient="vertical",
    command=tree.yview
)
scrollbar.pack(side="right", fill="y")
```

```
tree.configure(yscrollcommand=scrollbar.set)

def refresh_table():
    for item in tree.get_children():
        tree.delete(item)

    for record in get_filmography():
        # Use the database movie_id as the row ID, so duplicate movie titles
        # can still be selected and deleted correctly.
        tree.insert("", tk.END, iid=str(record[0]), values=record[1:])

def refresh_dropdowns():
    movie_director["values"] = get_director_options()
    box_movie["values"] = get_movie_options()

    if movie_director["values"]:
        movie_director.current(0)

    if box_movie["values"]:
        box_movie.current(0)

# -------------------- EDIT RECORDS WINDOW --------------------

def open_edit_window():

    edit_window = tk.Toplevel(root)
    edit_window.title("Edit Database Records")
    edit_window.geometry("520x540")
    edit_window.configure(bg=WHITE)

    tk.Label(
        edit_window,
        text="EDIT DATABASE RECORDS",
        bg=WHITE,
        fg=NAVY,
        font=("Arial", 16, "bold")
    ).pack(pady=20)

    form = tk.Frame(edit_window, bg=WHITE)
    form.pack(padx=25, pady=10, fill="x")

    make_label(form, "Record Type").grid(
        row=0, column=0, sticky="w", pady=8
    )

    record_type = ttk.Combobox(
        form,
        values=["Director", "Movie", "Box Office"],
        state="readonly",
        width=25
    )
    record_type.grid(row=0, column=1, padx=15, pady=8)
    record_type.current(0)

    make_label(form, "Record ID").grid(
        row=1, column=0, sticky="w", pady=8
```

```
)

record_id = make_entry(form)
record_id.grid(row=1, column=1, padx=15, pady=8)


fields_frame = tk.Frame(edit_window, bg=WHITE)
fields_frame.pack(padx=25, pady=10, fill="x")


entries = {}


def build_fields():
    for widget in fields_frame.winfo_children():
        widget.destroy()


    entries.clear()


    selected = record_type.get()


    if selected == "Director":
        fields = ["Name", "Nationality"]
    elif selected == "Movie":
        fields = ["Director ID", "Title", "Release Year"]
    else:
        fields = ["Movie ID", "Revenue (Millions)", "Territory"]


    for row, field in enumerate(fields):
        make_label(fields_frame, field).grid(
            row=row, column=0, sticky="w", pady=8
        )


        entry = make_entry(fields_frame)
        entry.grid(row=row, column=1, padx=15, pady=8)
        entries[field] = entry


def load_record():
    try:
        rid = int(record_id.get())
        selected = record_type.get()


        if selected == "Director":
            cursor.execute(
                "SELECT name, nationality FROM directors WHERE director_id=?",
                (rid,)
            )
        elif selected == "Movie":
            cursor.execute(
                "SELECT director_id, title, release_year FROM movies WHERE movie_id=?",
                (rid,)
            )
        else:
            cursor.execute(
                """SELECT movie_id, revenue_millions, territory
                   FROM box_office WHERE record_id=?""",
                (rid,)
            )


        data = cursor.fetchone()
```

```
        if data is None:
            messagebox.showwarning(
                "Not Found",
                "No record exists with this ID.",
                parent=edit_window
            )
            return

        for entry, value in zip(entries.values(), data):
            entry.delete(0, tk.END)
            entry.insert(0, str(value))

    except ValueError:
        messagebox.showerror(
            "Invalid ID",
            "Enter a numeric record ID.",
            parent=edit_window
        )

def save_record():
    try:
        rid = int(record_id.get())
        selected = record_type.get()
        values = [entry.get().strip() for entry in entries.values()]

        if any(value == "" for value in values):
            messagebox.showwarning(
                "Input Required",
                "Complete all fields.",
                parent=edit_window
            )
            return

        if selected == "Director":
            update_director(rid, values[0], values[1])

        elif selected == "Movie":
            year_value = int(values[2])
            if year_value <= 0:
                raise ValueError

            update_movie(
                rid,
                int(values[0]),
                values[1],
                year_value
            )

        else:
            revenue = float(values[1])

            if revenue < 0:
                raise ValueError

            update_box_office(
                rid,
                int(values[0]),
                revenue,
```

```
                    values[2]
                )

            messagebox.showinfo(
                "Success",
                "Record updated successfully.",
                parent=edit_window
            )

            refresh_dropdowns()
            refresh_table()
            edit_window.destroy()

        except ValueError:
            messagebox.showerror(
                "Invalid Input",
                "Check the IDs, year and revenue values.",
                parent=edit_window
            )
        except sqlite3.IntegrityError:
            messagebox.showerror(
                "Database Error",
                "The referenced Director ID or Movie ID does not exist.",
                parent=edit_window
            )
        except Exception as error:
            messagebox.showerror("Error", str(error), parent=edit_window)

    record_type.bind("<<ComboboxSelected>>", lambda event: build_fields())

    make_button(
        form,
        "LOAD RECORD",
        load_record
    ).grid(row=2, column=1, pady=15, sticky="w")

    build_fields()

    make_button(
        edit_window,
        "SAVE CHANGES",
        save_record
    ).pack(pady=20)

# -------------------- DELETE RECORDS --------------------

def delete_selected_record():
    selected = tree.selection()

    if not selected:
        messagebox.showwarning(
            "Select Record",
            "Select a filmography row first."
        )
        return

    values = tree.item(selected[0], "values")
    movie_title_value = values[1]
    movie_id = int(selected[0])
```

```
    confirm = messagebox.askyesno(
        "Confirm Delete",
        f"Delete '{movie_title_value}' and its box office records?"
    )

    if confirm:
        try:
            delete_movie(movie_id)
            messagebox.showinfo("Success", "Movie deleted successfully.")
            refresh_table()
            refresh_dropdowns()
        except Exception as error:
            messagebox.showerror("Error", str(error))

# -------------------- ACTION BUTTONS --------------------

action_frame = tk.Frame(records_tab, bg=WHITE)
action_frame.pack(pady=15)

make_button(
    action_frame,
    "EDIT RECORDS",
    open_edit_window
).pack(side="left", padx=10)

make_button(
    action_frame,
    "DELETE SELECTED MOVIE",
    delete_selected_record,
    color="#DC2626"
).pack(side="left", padx=10)

make_button(
    action_frame,
    "REFRESH TABLE",
    refresh_table,
    color="#475569"
).pack(side="left", padx=10)

# -------------------- INITIAL REFRESH --------------------

refresh_dropdowns()
refresh_table()

# -------------------- RUN APPLICATION --------------------

def close_application():
    # Commit pending changes and close the database connection.
    try:
        database.commit()
        database.close()
    finally:
        root.destroy()



root.protocol("WM_DELETE_WINDOW", close_application)
```

```
root.mainloop()
```

Conclusion

The Cinema Analytics application stores director, movie and box-office details in three related tables. It allows users to add, view, edit and delete records through a simple graphical interface. Primary keys and foreign keys connect the tables and help keep the data organised. Input checks help reduce incorrect entries. The JOIN operation displays related details together in one filmography table.

The project shows how Python, Tkinter, SQLite and SQL can work together to build a useful database application. Python manages the program and the interface, while SQLite stores the information and SQL handles the database operations.

Interpretation

This CE4 project applies database concepts to a cinema-related task. The information is divided into three tables: Directors, Movies and Box Office. Each table stores a different type of information, and the tables are connected using primary keys and foreign keys. This helps keep the information organised.

The application performs the main database tasks: creating, reading, updating and deleting records. It also uses JOIN to display information from more than one table together. The Tkinter window makes these tasks easier because users can enter and manage information through forms and buttons. In simple terms, the project shows how a database can be planned, connected and used through a working application.